BookTrendsRecSys

Helping readers pick their next book — fast, fair, and useful

Workshop 2

October 16, 2025

Agenda

- The problem we solve
- What you get (benefits in plain language)
- Quick demo preview
- Trust & fairness — in simple terms
- Where this fits today
- What is next and how to try it

The Problem

- **Too many options**: for most readers, choosing a next book takes time.
- **Not sure what to trust**: ratings are messy and popularity can be misleading.
- **Staff time is limited**: librarians and curators need quick shortlists.

Our Promise

- **A clean Top-10 list** per user — titles you can actually act on.
- **At a glance insights**: why these books appear (based on past likes).
- **Fair experience**: we watch for gaps across regions and genres.

What You Get (Benefits)

- **Save time**: fewer clicks, faster choices.
- **Better fit**: suggestions follow what the reader enjoyed before.
- **Explainable**: simple visuals to see patterns in ratings.
- **Portable**: runs as a small app; no special hardware needed.

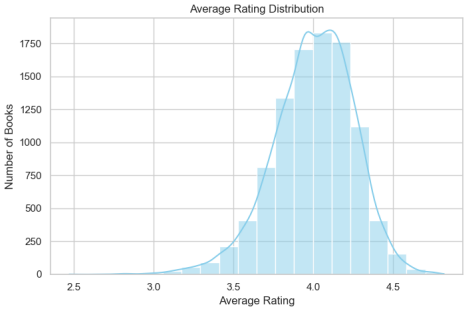
One-Minute View of How It Works

Behind the scenes

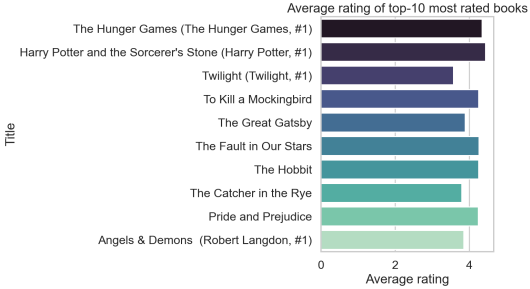
Your past ratings → we learn look-alike tastes → score many books → show the Top-10.

- Think of it as "people who liked X also liked Y" made practical.
- If the list is empty, we fall back to helpful, popular picks.

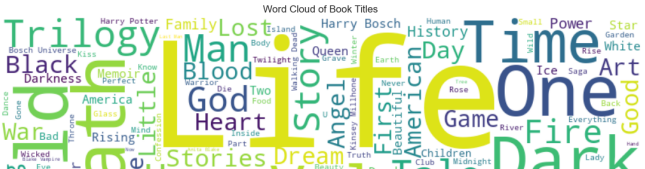
Demo Preview



Ratings at a glance

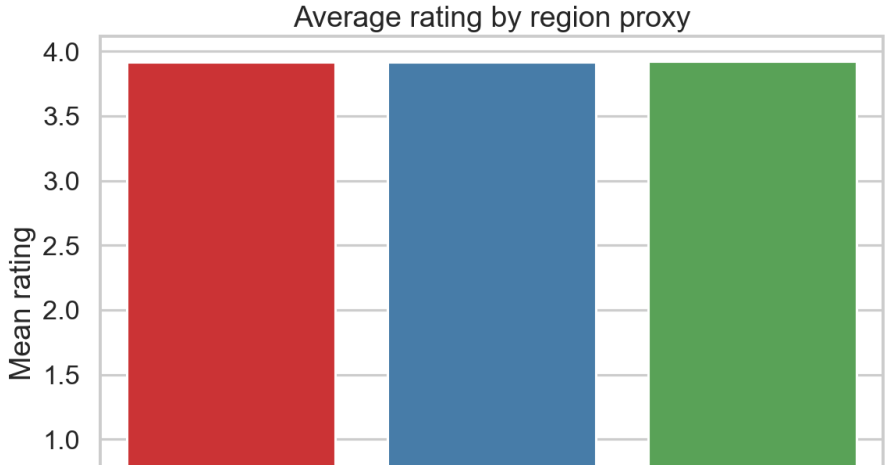


Popular vs. quality overview



Fairness & Trust

- **Goal**: similar experience for readers in different groups.
- **What we check**: how often the list is useful across regions or genres.
- **Today**: small gaps; we keep monitoring as data changes.



Who Benefits

- **Readers**: quick, relevant ideas when they open the app.
- **Librarians/Curators**: ready-to-share shortlists.
- **Program managers**: basic fairness snapshot without extra tooling.

Where It Fits

- **Front page module**: “Because you liked...” Top-10.
- **Kiosk/library desk**: fast suggestions during patron support.
- **Newsletter picks**: plug the list into monthly recommendations.

Limitations

- Works best if the reader has **some** rating history.
- Sparse genres may need **fallback** to popularity.
- We focus on relevance first; **deep explanations** are in progress.

What Is Next

- **Smarter cold-start**: better picks for new users.
- **Genre-aware fallback**: keep variety while staying relevant.
- **Simple reasons**: short labels like “Because you liked *Dune*”.

How to Try It

- Open the app, enter a user ID, and (optionally) a genre.
- Review the Top-10 and the simple insights view.
- Share feedback: did this save time? did it feel relevant?

Thank You

- Questions and ideas welcome.
- Want a short pilot? We can set one up quickly.

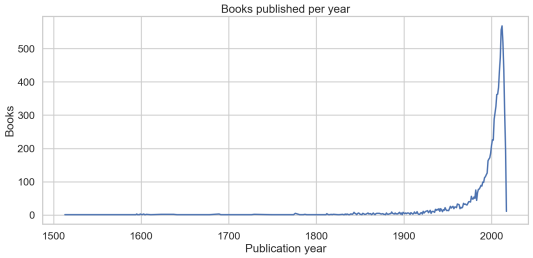
Quick Evaluation Snapshot (Appendix)

- Precision@10: 0.0240% Recall@10: 0.0204% NDCG@10: 0.0249%
- Validation NDCG@10: 0.0148%
- Region gap (NDCG@10): 0.0052 pp Genre gap: 0.0786 pp

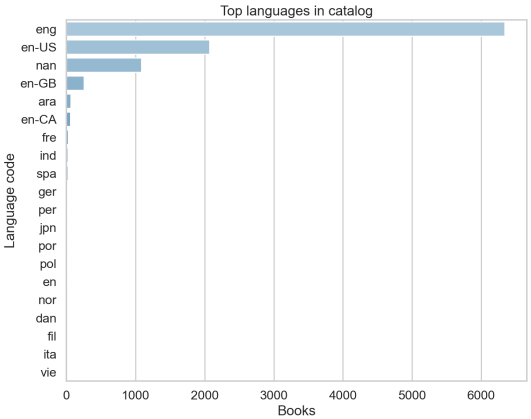
Under the Hood (Appendix)

- Data: public ratings; train/validation split.
- Model: implicit ALS (`rank`=64, `reg`=0.05, `alpha`=20, `iters`=10).
- App: small Streamlit front end with a simple insights tab.

Extra Visuals (Appendix)



Books per year



Top languages

(Backup) Ranking Metrics — Formulas

For user u with ground truth G_u and top-K list $R_u = [i_1, \ldots, i_K]$:

$$\mathrm{P@K} = \frac{1}{|U|} \sum_u \frac{|R_u \cap G_u|}{K}, \quad \mathrm{R@K} = \frac{1}{|U|} \sum_u \frac{|R_u \cap G_u|}{|G_u|}$$

$$\mathrm{DCG@K}(u) = \sum_{j=1}^{K} \frac{\mathbf{1}[i_j \in G_u]}{\log_2(j+1)}, \quad \mathrm{NDCG@K} = \frac{1}{|U|} \sum_u \frac{\mathrm{DCG@K}(u)}{\mathrm{IDCG@K}(u)}$$

(Backup) Minimal File Map

```
/scripts
|-- get_rating_of_users.py
|-- get_rating_of_books.py
|-- build_goodreads_dataset.py
|-- count_books.py
‘-- goodreads_dataset.csv
/src
|-- data_prep.py  |-- als_train.py
|-- eval_ranking.py  |-- parity.py
/figs/eda, /models, /metrics, /data/processed
```